

TP 2 - Régression linéaire

M2 IWOCS, Apprentissage Automatique

Prénom et nom : Hamadou BA

Exercice 1 - Régression linéaire univarié

Les données utilisées pour cet exercice décrivent les tailles d'enfants d'âge compris entre 2 et 8 ans : le fichier `ex1x.dat` correspond à leur âge et `ex1y.dat` correspond à leur taille (en mètres). On a dans ces 2 fichiers, les données de 50 enfants rangées dans le même ordre.

Ces données constituent des exemples d'apprentissage qui vont être utilisées afin de construire un modèle de régression linéaire qui a pour objectif de prédire la taille d'un enfant à partir de son âge.

```
In [1]: import numpy as np
import matplotlib.pyplot as plt
from sklearn.preprocessing import StandardScaler
```

1. Tout d'abord, nous allons utiliser la fonction `loadtxt` du package `numpy` en Python (ici désigné par `np`). Cette fonction permet de lire les fichiers et de les convertir en vecteurs/matrices :

```
x=np.loadtxt('ex1x.dat');
y=np.loadtxt('ex1y.dat');
```

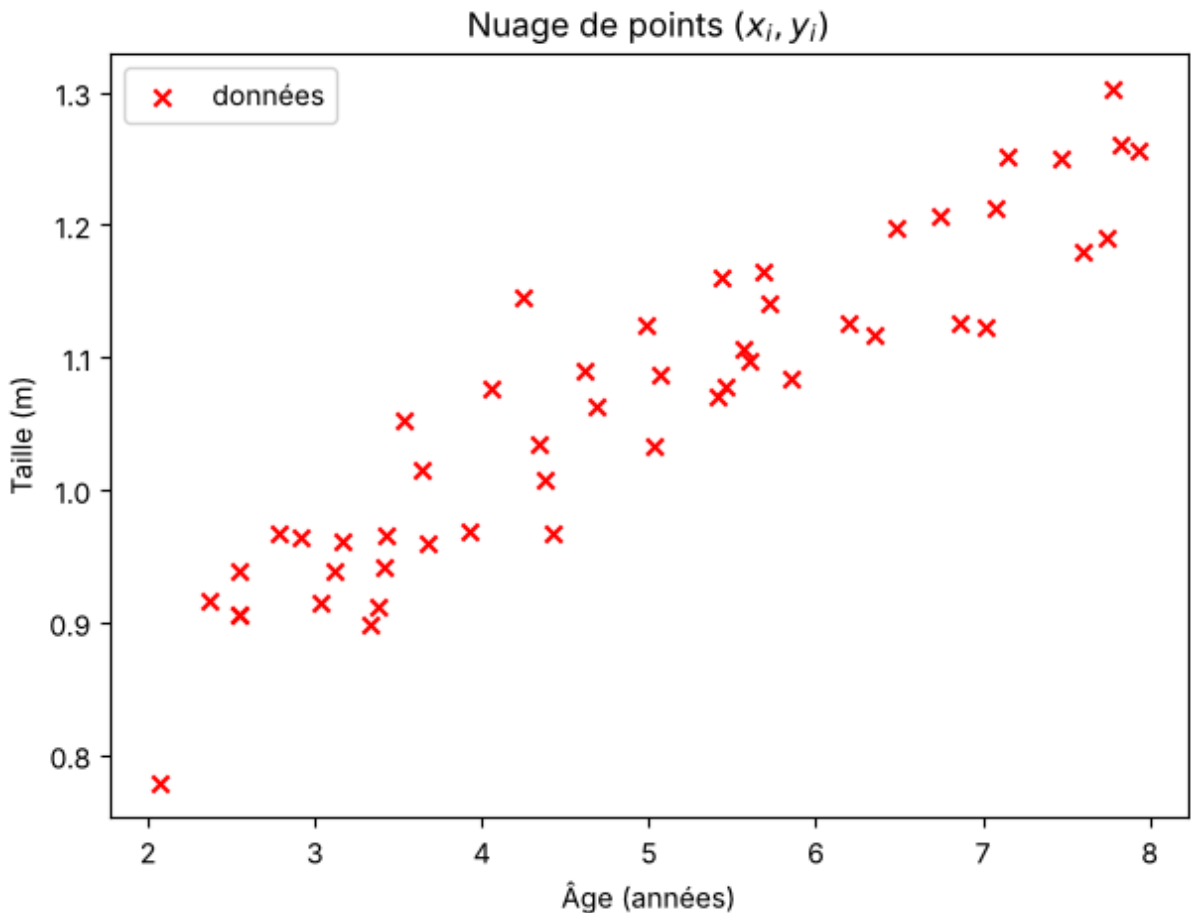
```
In [2]: x = np.loadtxt('ex1x.dat')
y = np.loadtxt('ex1y.dat')
m = len(x) # nombre d'exemples d'apprentissage
print('x :', x.shape, ' y :', y.shape)
print('âges :', x[:5])
print('tailles :', y[:5])
```

```
x : (50,) y : (50,)
âges : [2.0658746 2.3684087 2.5399929 2.5420804 2.549079 ]
tailles : [0.77918926 0.91596757 0.90538354 0.90566138 0.9389889 ]
```

Réponse : on obtient deux vecteurs de $m = 50$ valeurs : les âges x (entre 2 et 8 ans) et les tailles y (en mètres), rangés dans le même ordre.

2. Afficher le nuage de points (x_i, y_i) .

```
In [3]: plt.figure(figsize=(7, 5))
plt.scatter(x, y, c='r', marker='x', label='données')
plt.xlabel('Âge (années)')
plt.ylabel('Taille (m)')
plt.title('Nuage de points $(x_i, y_i)$')
plt.legend()
plt.show()
```



Réponse : les points s'alignent approximativement le long d'une droite croissante : la taille augmente avec l'âge. Un modèle de régression linéaire est donc adapté.

3. Définir en Python, la fonction hypothèse correspondant à un modèle de régression linéaire. Soit le vecteur $\theta = (\theta_0, \theta_1)$, cette fonction hypothèse s'écrit

$$h(\theta_0, \theta_1, x) = \theta_0 + \theta_1 x$$

```
In [4]: def h(theta0, theta1, x):
        """Fonction hypothèse :  $h(\theta_0, \theta_1, x) = \theta_0 + \theta_1 * x$ """
        return theta0 + theta1 * x
```

Réponse : h fonctionne aussi bien pour un réel que pour un vecteur `numpy` x (calcul composante par composante).

4. Définir en Python, la fonction de coût $J(\theta_0, \theta_1)$ telle que vue en cours.

```
In [5]: def J(theta0, theta1):  
        """Fonction de coût (critère des moindres carrés) : J = 1/(2m) * somme  
        (h(x_i) - y_i)^2"""  
        return np.sum((h(theta0, theta1, x) - y) ** 2) / (2 * m)  
  
        print('J(0, 0) =', J(0, 0))
```

J(0, 0) = 0.5737324907367211

Réponse : $J(\theta_0, \theta_1) = \frac{1}{2m} \sum_{i=1}^m (\theta_0 + \theta_1 x_i - y_i)^2$. Au point de départ $\theta = (0, 0)$, on a $J \approx 0,574$.

5. Définir en Python, une fonction qui réalise une itération et qui va renvoyer θ_0^* et θ_1^* , en fonction de θ_0 et θ_1 , selon les formules de descente de gradient vue en cours.

Pour faire fonctionner cette méthode de gradient, il faut définir la valeur du coefficient d'apprentissage noté α dans le cours ; il régle la profondeur de descente. On propose ici que ce coefficient prenne une valeur constante égale à 0,07. On partira aussi des valeurs initiales $\theta_0 = \theta_1 = 0$.

```
In [6]: alpha = 0.07 # coefficient d'apprentissage  
  
def iteration(theta0, theta1):  
    """Une itération de la descente de gradient : renvoie (theta0*,  
    theta1*)"""  
    e = h(theta0, theta1, x) - y # écarts h(x_i) - y_i  
    theta0_star = theta0 - alpha / m * np.sum(e)  
    theta1_star = theta1 - alpha / m * np.sum(e * x)  
    return theta0_star, theta1_star # mise à jour simultanée  
  
print(iteration(0, 0))
```

(np.float64(0.07452802382200001), np.float64(0.38002167250780633))

Réponse : d'après le cours, $\theta_0^* = \theta_0 - \frac{\alpha}{m} \sum_{i=1}^m (h_{\theta}(x_i) - y_i)$ et

$\theta_1^* = \theta_1 - \frac{\alpha}{m} \sum_{i=1}^m (h_{\theta}(x_i) - y_i) x_i$. Les deux nouvelles valeurs sont calculées à

partir des **anciennes** valeurs de θ_0 et θ_1 (mise à jour simultanée), puis on remplace θ par θ^* à l'itération suivante.

6. Faire tourner la méthode de descente de gradient sur quelques itérations puis représenter la droite

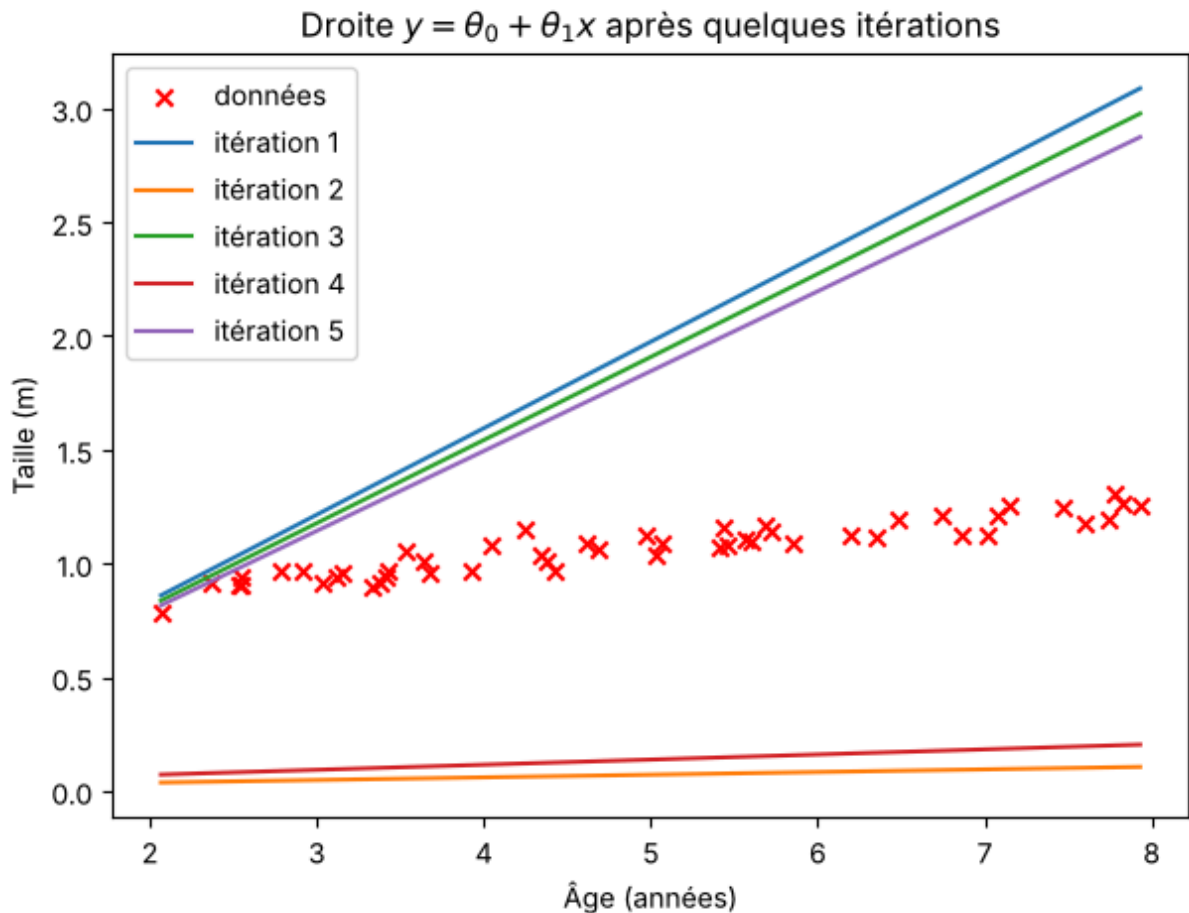
$$y = \theta_0 + \theta_1 x$$

sur le nuage de points.

```
In [7]: theta0, theta1 = 0.0, 0.0
xs = np.linspace(x.min(), x.max(), 100)

plt.figure(figsize=(7, 5))
plt.scatter(x, y, c='r', marker='x', label='données')
for k in range(1, 6):
    theta0, theta1 = iteration(theta0, theta1)
    print(f'itération {k} : theta0 = {theta0:.4f}, theta1 = {theta1:.4f}, J = {J(theta0, theta1):.4f}')
    plt.plot(xs, h(theta0, theta1, xs), label=f'itération {k}')
plt.xlabel('Âge (années)'); plt.ylabel('Taille (m)')
plt.title('Droite $y=\theta_0+\theta_1 x$ après quelques itérations')
plt.legend(); plt.show()
```

```
itération 1 : theta0 = 0.0745, theta1 = 0.3800, J = 0.5351
itération 2 : theta0 = 0.0129, theta1 = 0.0117, J = 0.4992
itération 3 : theta0 = 0.0825, theta1 = 0.3650, J = 0.4658
itération 4 : theta0 = 0.0254, theta1 = 0.0225, J = 0.4347
itération 5 : theta0 = 0.0904, theta1 = 0.3510, J = 0.4057
```



Réponse : après 5 itérations, J a diminué ($0,574 \rightarrow 0,406$) mais la droite n'est pas encore ajustée. On observe que θ_1 **oscille** : il vaut environ 0,38 aux itérations impaires (droite beaucoup trop pentue) et environ 0,01 aux itérations paires (droite presque horizontale, trop basse), tandis que θ_0 augmente lentement ($\approx 0,09$).

Avec $\alpha = 0,07$ et des âges allant jusqu'à 8, le pas est assez grand dans la direction θ_1 : la descente « rebondit » d'un bord à l'autre de la vallée de J , tout en descendant. Il faut poursuivre les itérations pour converger.

7. Faire tourner la méthode de descente de gradient pendant toutes les itérations nécessaires à faire converger la solution θ recherchée. Pour cela on définit le critère d'arrêt du processus itératif par

$$\left| \frac{J(\theta^*) - J(\theta)}{J(\theta)} \right| < 10^{-3}$$

Afficher la droite obtenue suite à cette convergence sur le nuage de points.

```

In [8]: eps = 1e-3 # critère d'arrêt imposé par l'énoncé

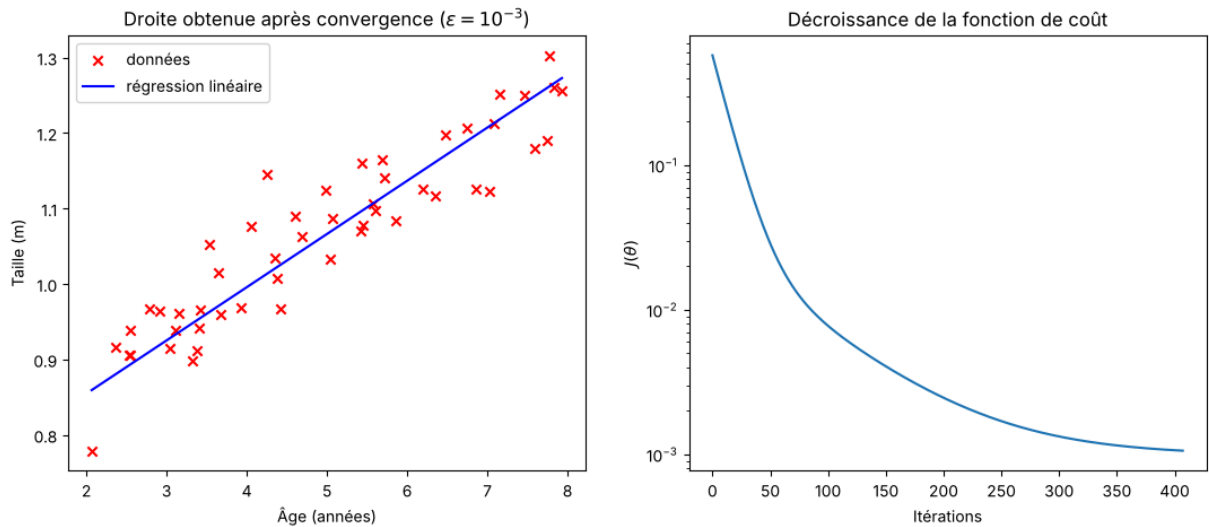
def descente(eps, theta0=0.0, theta1=0.0):
    """Itère jusqu'à  $|J(\theta^*) - J(\theta)| / J(\theta) < \text{eps}$  ; renvoie
    theta0, theta1, nb d'itérations, historique de J"""
    hist = [J(theta0, theta1)]
    while True:
        theta0, theta1 = iteration(theta0, theta1)
        hist.append(J(theta0, theta1))
        if abs((hist[-1] - hist[-2]) / hist[-2]) < eps:
            return theta0, theta1, len(hist) - 1, hist

theta0, theta1, n_iter, hist = descente(eps)
print("nombre d'itérations :", n_iter)
print(f'theta0 = {theta0:.4f}    theta1 = {theta1:.4f}    J = {J(theta0,
theta1):.6f}')

fig, ax = plt.subplots(1, 2, figsize=(13, 5))
ax[0].scatter(x, y, c='r', marker='x', label='données')
ax[0].plot(xs, h(theta0, theta1, xs), 'b', label='régression linéaire')
ax[0].set_xlabel('Âge (années)'); ax[0].set_ylabel('Taille (m)')
ax[0].set_title('Droite obtenue après convergence
( $\epsilon=10^{-3}$ )'); ax[0].legend()
ax[1].plot(hist)
ax[1].set_yscale('log'); ax[1].set_xlabel('Itérations');
ax[1].set_ylabel('$J(\theta)$')
ax[1].set_title('Décroissance de la fonction de coût')
plt.show()

```

nombre d'itérations : 407
theta0 = 0.7139 theta1 = 0.0705 J = 0.001058



Réponse : avec le critère d'arrêt $\left| \frac{J(\theta^*) - J(\theta)}{J(\theta)} \right| < 10^{-3}$, l'algorithme s'arrête après **407 itérations** avec $\theta_0 \approx 0,714$ et $\theta_1 \approx 0,0705$. La droite passe bien au milieu du nuage de points et J décroît de façon monotone.

Interprétation : $\theta_1 \approx 0,07$ signifie qu'un enfant grandit d'environ 7 cm par an sur cette tranche d'âge.

Remarque sur la précision du critère d'arrêt. Le critère porte sur la variation *relative* de J entre deux itérations, pas sur la distance à l'optimum. Comme la « vallée » de J est très allongée (voir question 9), la décroissance est lente en fin de calcul et le critère 10^{-3} arrête l'algorithme un peu avant le minimum. Le tableau ci-dessous compare avec des seuils plus petits et avec la solution exacte des moindres carrés (`np.polyfit`) :

```
In [9]: print(f"{'epsilon':>8} | {'itérations':>10} | {'theta0':>8} | {'theta1':>8} | {'J':>10}")
for e in (1e-3, 1e-6, 1e-10):
    t0, t1, n, _ = descente(e)
    print(f'{e:>8.0e} | {n:>10} | {t0:>8.4f} | {t1:>8.4f} | {J(t0, t1):>10.7f}')
t1_ex, t0_ex = np.polyfit(x, y, 1)
print(f"{'exacte':>8} | {'-':>10} | {t0_ex:>8.4f} | {t1_ex:>8.4f} | {J(t0_ex, t1_ex):>10.7f}")
```

epsilon	itérations	theta0	theta1	J
1e-03	407	0.7139	0.0705	0.0010580
1e-06	883	0.7490	0.0641	0.0009871
1e-10	1512	0.7502	0.0639	0.0009871
exacte	-	0.7502	0.0639	0.0009871

Réponse : plus ε est petit, plus θ se rapproche de la solution exacte (0,7502; 0,0639), au prix de plus d'itérations. Avec $\varepsilon = 10^{-3}$, le coût J n'est que 7 % au-dessus du minimum : la droite est déjà très bonne. Pour la suite, on garde le θ obtenu avec le critère de l'énoncé (10^{-3}).

8. On peut désormais utiliser le modèle pour faire des prédictions : quelle est la taille de 3 enfants d'âges respectifs 3, 5 et 7 ans ?

```
In [10]: for age in (3, 5, 7):
    print(f'âge {age} ans -> taille prédite : {h(theta0, theta1, age):.3f} m')
```

```
âge 3 ans -> taille prédite : 0.925 m
âge 5 ans -> taille prédite : 1.066 m
âge 7 ans -> taille prédite : 1.207 m
```

Réponse : avec $\theta = (0,714; 0,0705)$:

Âge	Taille prédite
3 ans	0,925 m
5 ans	1,066 m
7 ans	1,207 m

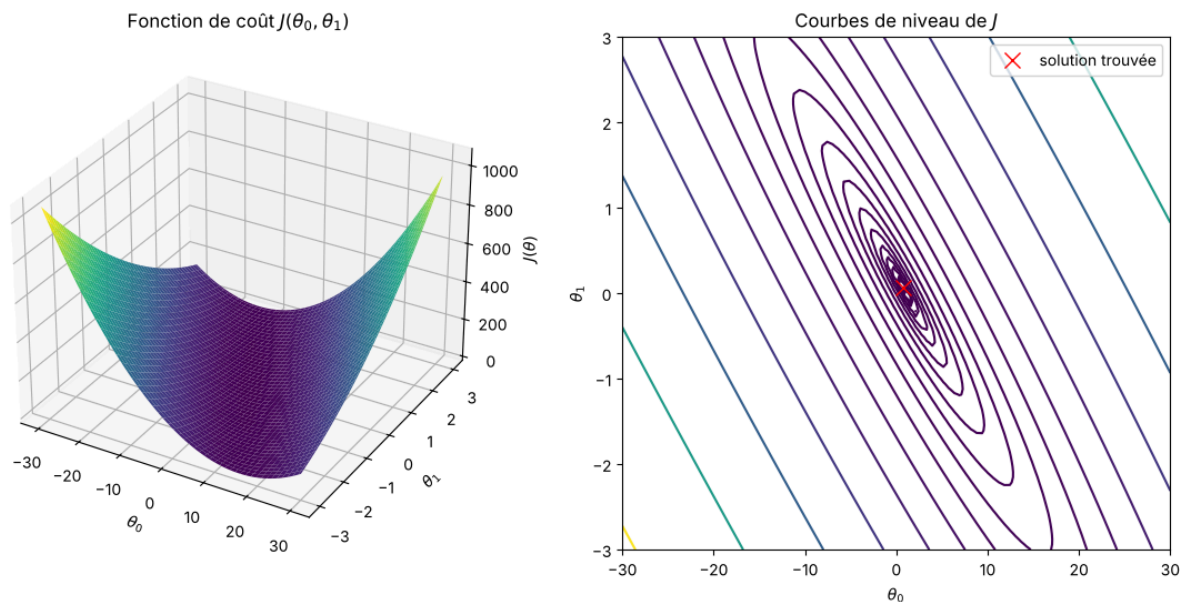
(Avec la solution exacte, on obtiendrait 0,942 m, 1,070 m et 1,197 m : l'écart reste de l'ordre du centimètre.)

9. Nous allons maintenant visualiser la fonction de coût $J(\theta)$ en 3D sur une grille de base 100×100 et des valeurs pour θ dans les intervalles suivants (il faudra donc prendre 100 valeurs réparties dans ces intervalles) : $\theta_0 \in [-30; 30]$ et $\theta_1 \in [-3; 3]$.

```
In [11]: theta0_vals = np.linspace(-30, 30, 100)
theta1_vals = np.linspace(-3, 3, 100)
J_vals = np.zeros((100, 100))
for i, t0 in enumerate(theta0_vals):
    for j, t1 in enumerate(theta1_vals):
        J_vals[j, i] = J(t0, t1)           # ligne : theta1, colonne :
theta0
T0, T1 = np.meshgrid(theta0_vals, theta1_vals)

fig = plt.figure(figsize=(14, 6))
ax = fig.add_subplot(1, 2, 1, projection='3d')
ax.plot_surface(T0, T1, J_vals, cmap='viridis')
ax.set_xlabel('$\\theta_0$'); ax.set_ylabel('$\\theta_1$');
ax.set_zlabel('$J(\\theta)$')
ax.set_title('Fonction de coût $J(\\theta_0, \\theta_1)$')

ax2 = fig.add_subplot(1, 2, 2)
cs = ax2.contour(T0, T1, J_vals, levels=np.logspace(-2, 3, 20))
ax2.plot(theta0, theta1, 'rx', markersize=10, label='solution trouvée')
ax2.set_xlabel('$\\theta_0$'); ax2.set_ylabel('$\\theta_1$')
ax2.set_title('Courbes de niveau de $J$'); ax2.legend()
plt.show()
```



Réponse : la surface $J(\theta_0, \theta_1)$ a la forme d'une **cuvette** (paraboloïde) : la fonction de coût des moindres carrés est convexe et possède un **unique minimum**, situé au point trouvé par la descente de gradient (croix rouge). Il n'y a donc pas de minimum local piège : la descente de gradient converge vers le minimum global.

Les courbes de niveau sont des ellipses très allongées : J varie beaucoup selon θ_1 et très peu selon θ_0 . C'est pour cela que la convergence est lente (des centaines d'itérations) et qu'un critère d'arrêt relatif comme 10^{-3} s'arrête un peu avant le fond de la vallée.

Exercice 2 - Régression linéaire multivariée et écriture matricielle de la méthode de descente du gradient

Dans ce problème, on utilise des données correspondant à 47 exemples d'apprentissage sur des données immobilières à Portland, Oregon (USA). Les données d'entrée x (stockées dans le fichier `ex2x.dat`) correspondent aux surfaces et au nombre de pièces de ces 47 appartements et la donnée cible y (stockée dans le fichier `ex2y.dat`) correspond au prix de ces mêmes appartements.

1. Utiliser la fonction `loadtxt` du package `numpy` en Python (ici désigné par `np`). Cette fonction permet de lire les fichiers et de les convertir en vecteurs/matrices :

```
x=np.loadtxt('ex2x.dat');
y=np.loadtxt('ex2y.dat');
```

```
In [12]: x = np.loadtxt('ex2x.dat')
y = np.loadtxt('ex2y.dat')
m = len(y)
print('x :', x.shape, ' y :', y.shape)
print('5 premiers logements (surface, pièces) -> prix :')
for xi, yi in zip(x[:5], y[:5]):
    print('    ', xi, '->', yi)
```

```
x : (47, 2) y : (47,)
5 premiers logements (surface, pièces) -> prix :
[2104.    3.] -> 399900.0
[1600.    3.] -> 329900.0
[2400.    3.] -> 369000.0
[1416.    2.] -> 232000.0
[3000.    4.] -> 539900.0
```

Réponse : x est une matrice 47×2 (colonne 1 : surface, colonne 2 : nombre de pièces) et y un vecteur de 47 prix.

2. Prétraitement des données : utiliser la fonction `sklearn.preprocessing.StandardScaler` pour normaliser les données.

```
In [13]: print('avant normalisation : moyennes =', x.mean(axis=0), ' écarts-types
          =', x.std(axis=0))

          scaler = StandardScaler()                # normalisation "centrage-réduction"
          x_norm = scaler.fit_transform(x)
          print('après normalisation : moyennes =', x_norm.mean(axis=0).round(10), '
          écarts-types =', x_norm.std(axis=0))

          # matrice augmentée X (colonne de 1 pour theta0) et vecteur cible Y
          X = np.hstack([np.ones((m, 1)), x_norm])
          Y = y.reshape(-1, 1)
          print('X :', X.shape, ' Y :', Y.shape)

avant normalisation : moyennes = [2000.68085106    3.17021277] écarts-types
= [7.86202619e+02 7.52842809e-01]
après normalisation : moyennes = [-0.  0.] écarts-types = [1. 1.]
X : (47, 3) Y : (47, 1)
```

Réponse : `StandardScaler` applique la normalisation « centrage-réduction » du cours : $\overline{X}_{ij} = \frac{X_{ij} - \mu(X_i)}{\sigma(X_i)}$. Chaque colonne a alors une moyenne nulle et un écart-type égal à 1.

C'est indispensable ici : la surface vaut environ 2000 alors que le nombre de pièces vaut environ 3. Sans normalisation, les deux paramètres ont des échelles très différentes et la descente de gradient converge très mal (ou diverge pour $\alpha = 0,07$).

On construit ensuite la matrice augmentée $X = [I_1 \ X_1 \ X_2]$ (colonne de 1 en premier) et le vecteur colonne Y .

3. Définir les fonctions en Python sous forme matricielle pour représenter la fonction hypothèse $h_\theta(X)$, le vecteur tel que défini dans le cours : $E = h_\theta(X) - Y$, la fonction décrivant une itération et la fonction de coût $J(\theta)$.

```
In [14]: def h(theta, X):
    """Hypothèse sous forme matricielle : h_theta(X) = X theta"""
    return X @ theta

def E(theta, X, Y):
    """Vecteur des écarts : E = h_theta(X) - Y"""
    return h(theta, X) - Y

def J(theta, X, Y):
    """Fonction de coût : J(theta) = 1/(2m) E^t E"""
    e = E(theta, X, Y)
    return (e.T @ e).item() / (2 * len(Y))

def iteration(theta, alpha, X, Y):
    """Une itération : theta* = theta - alpha/m X^t E"""
    return theta - alpha / len(Y) * X.T @ E(theta, X, Y)

theta = np.zeros((3, 1))
print('J(0) =', J(theta, X, Y))
```

J(0) = 65591548106.45744

Réponse : en notation matricielle (cours, §6.7) :

- hypothèse : $h_{\theta}(X) = X\theta$;
- écarts : $E = h_{\theta}(X) - Y$;
- coût : $J(\theta) = \frac{1}{2m} E^t E$;
- itération : $\theta^* = \theta - \frac{\alpha}{m} X^t E$.

Ces formules regroupent en un seul produit matriciel les $n + 1$ formules de mise à jour $\theta_j^* = \theta_j - \frac{\alpha}{m} \sum_i (h_{\theta}(x_i) - y_i) x_{ij}$.

4. Pour la valeur du taux d'apprentissage $\alpha = 0,07$, effectuer le calcul de régression permettant d'obtenir le vecteur $\theta = (\theta_0, \theta_1, \theta_2)$ optimal permettant de calculer la meilleure régression linéaire multivariée sur le jeu de données d'apprentissage.

```
In [15]: eps = 1e-3 # même critère d'arrêt que dans l'exercice 1

def descente(alpha, eps=eps, max_iter=100000):
    """Itère depuis theta = 0 jusqu'à |J(theta*) - J(theta)| / J(theta) <
    eps"""
    theta = np.zeros((X.shape[1], 1))
    hist = [J(theta, X, Y)]
    for _ in range(max_iter):
        theta = iteration(theta, alpha, X, Y)
        hist.append(J(theta, X, Y))
        if abs((hist[-1] - hist[-2]) / hist[-2]) < eps:
            break
    return theta, len(hist) - 1, hist

theta, n_iter, hist = descente(0.07)
print("nombre d'itérations :", n_iter)
print('theta =', theta.ravel().round(2))
print(f'J(theta) = {J(theta, X, Y):.4e}')
```

```
nombre d'itérations : 67
theta = [337780.19 102294.51    530.51]
J(theta) = 2.0691e+09
```

Réponse : avec $\alpha = 0,07$ et le critère 10^{-3} , la descente s'arrête après **67 itérations** avec $\theta \approx (337\,780; 102\,295; 531)$.

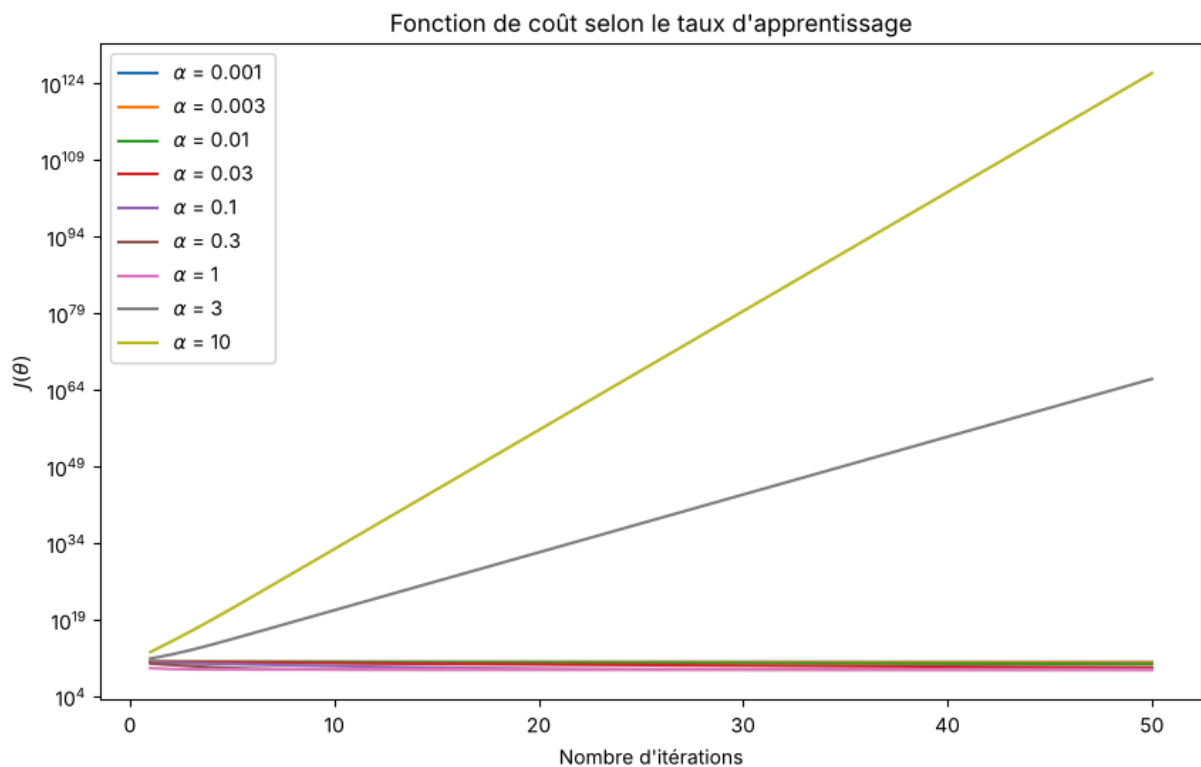
Comme les données sont normalisées, θ_0 correspond au prix d'un logement « moyen » (surface et nombre de pièces moyens), θ_1 à l'effet d'un écart-type de surface et θ_2 à l'effet d'un écart-type de nombre de pièces. Avec ce critère, θ n'est pas encore tout à fait à l'optimum (voir la comparaison à la question 5).

5. Nous allons maintenant automatiser la recherche du meilleur taux d'apprentissage $\alpha \in [0,001; 10]$. Pour ce faire, on devra calculer pour chaque itération la valeur de la fonction de coût $J(\theta)$ et on stockera toutes ces valeurs dans un vecteur. Comme on veut sélectionner un taux d'apprentissage efficace, on va comparer les résultats de calcul de $J(\theta)$ sur 50 itérations en changeant de taux d'apprentissage à chaque série d'itérations. Les valeurs de ce taux doivent rester dans l'intervalle initialement donné $\alpha \in [0,001; 10]$. On tracera alors les courbes représentant en abscisse, le nombre d'itérations et en ordonnées les valeurs de la fonction de coût ; chaque courbe correspond à une valeur du taux d'apprentissage. A partir de ces courbes, sélectionner ce qui paraît être le meilleur taux d'apprentissage et recalculer le vecteur θ jusqu'à la convergence. Utiliser ce vecteur θ pour prédire le prix d'un logement de 1650 m² et de 3 pièces.

```
In [16]: alphas = [0.001, 0.003, 0.01, 0.03, 0.1, 0.3, 1, 3, 10] # valeurs dans [0,001 ; 10]
n_it = 50

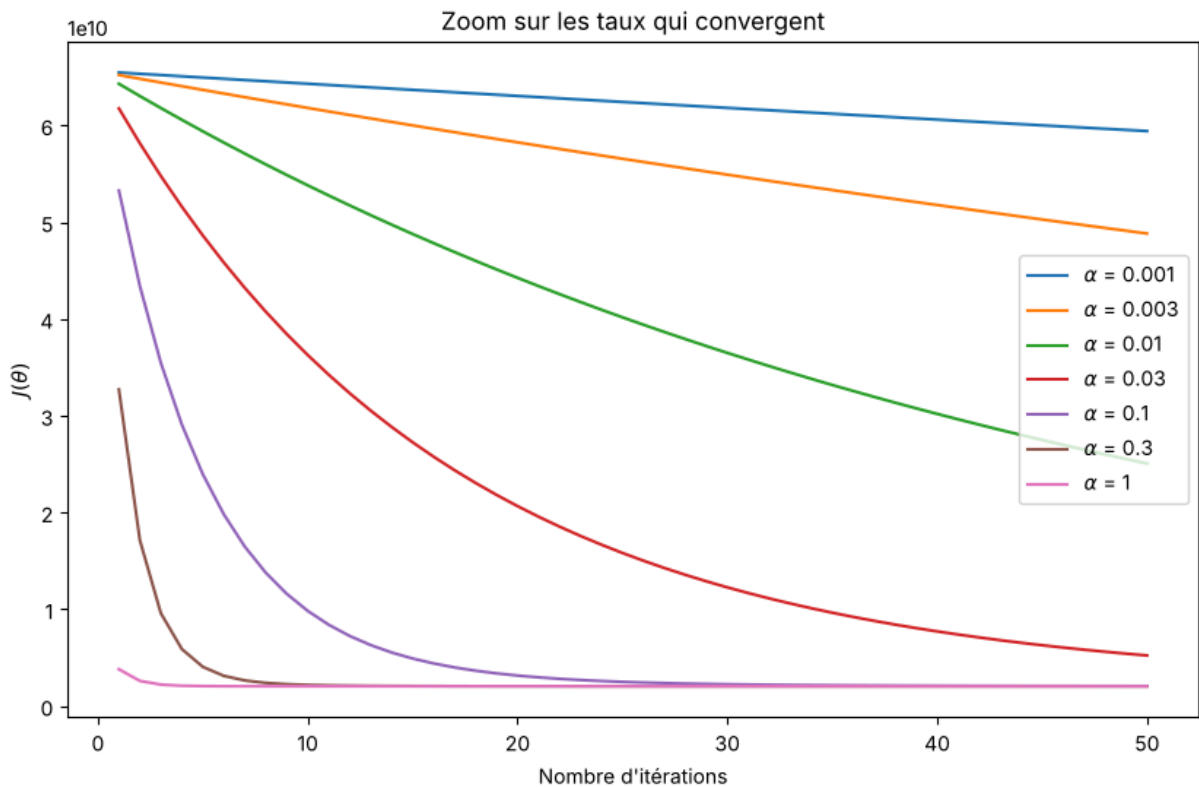
plt.figure(figsize=(10, 6))
for a in alphas:
    theta_a = np.zeros((3, 1))
    J_hist = np.zeros(n_it) # vecteur des valeurs de J
    for k in range(n_it):
        theta_a = iteration(theta_a, a, X, Y)
        J_hist[k] = J(theta_a, X, Y)
    print(f'alpha = {a:<6} J après 50 itérations = {J_hist[-1]:.4e}')
    plt.plot(range(1, n_it + 1), J_hist, label=f'$\\alpha$ = {a}')
plt.yscale('log')
plt.xlabel("Nombre d'itérations"); plt.ylabel('$J(\\theta)$')
plt.title("Fonction de coût selon le taux d'apprentissage")
plt.legend(); plt.show()
```

```
alpha = 0.001    J après 50 itérations = 5.9415e+10
alpha = 0.003    J après 50 itérations = 4.8827e+10
alpha = 0.01     J après 50 itérations = 2.5061e+10
alpha = 0.03     J après 50 itérations = 5.2248e+09
alpha = 0.1      J après 50 itérations = 2.0613e+09
alpha = 0.3      J après 50 itérations = 2.0433e+09
alpha = 1        J après 50 itérations = 2.0433e+09
alpha = 3        J après 50 itérations = 1.5822e+06
alpha = 10       J après 50 itérations = 1.1219e+126
```



Pour mieux distinguer les taux qui convergent, on retire les deux valeurs qui divergent :

```
In [17]: plt.figure(figsize=(10, 6))
for a in [0.001, 0.003, 0.01, 0.03, 0.1, 0.3, 1]:
    theta_a = np.zeros((3, 1)); J_hist = np.zeros(n_it)
    for k in range(n_it):
        theta_a = iteration(theta_a, a, X, Y)
        J_hist[k] = J(theta_a, X, Y)
    plt.plot(range(1, n_it + 1), J_hist, label=f'$\\alpha$ = {a}')
plt.xlabel("Nombre d'itérations"); plt.ylabel('$J(\\theta)$')
plt.title('Zoom sur les taux qui convergent')
plt.legend(); plt.show()
```



Réponse : lecture des courbes :

- $\alpha \in \{0,001; 0,003; 0,01\}$: J décroît trop lentement, on est encore loin du minimum après 50 itérations ;
- $\alpha \in \{0,03; 0,1\}$: convergence correcte mais encore lente ;
- $\alpha \in \{0,3; 1\}$: J atteint son minimum en quelques itérations seulement ;
- $\alpha \in \{3; 10\}$: le pas est trop grand, on « rate » le minimum et J **diverge** (après 50 itérations il atteint 10^{66} et 10^{126}).

Le meilleur taux d'apprentissage est donc $\alpha = 1$: c'est la décroissance la plus rapide sans divergence (avec le critère 10^{-3} , $\alpha = 1$ converge en 8 itérations contre 20 pour $\alpha = 0,3$ et 50 pour $\alpha = 0,1$). On recalcule θ jusqu'à la convergence avec cette valeur :

```
In [18]: alpha_opt = 1
theta_opt, n_opt, _ = descente(alpha_opt)
print("nombre d'itérations :", n_opt)
print('theta =', theta_opt.ravel().round(2))
print(f'J(theta) = {J(theta_opt, X, Y):.4e}')

# vérification : solution exacte des moindres carrés (équations normales
X^t X theta = X^t Y)
theta_exact = np.linalg.solve(X.T @ X, X.T @ Y)
print('solution exacte :', theta_exact.ravel().round(2))
```

```
nombre d'itérations : 8
theta = [340412.66 108389.74 -6514.76]
J(theta) = 2.0438e+09
solution exacte : [340412.66 109447.8 -6578.35]
```

Réponse : avec $\alpha = 1$, le critère 10^{-3} est atteint en **8 itérations** (contre 67 avec $\alpha = 0,07$) et on obtient $\theta \approx (340\,413; 108\,390; -6\,515)$, très proche de la solution exacte des moindres carrés $(340\,413; 109\,448; -6\,578)$ (écart relatif d'environ 1 % sur θ_1 , alors qu'avec $\alpha = 0,07$ on était encore à 7 % de l'optimum). Le grand taux d'apprentissage permet donc à la fois une convergence plus rapide **et** un résultat plus précis pour le même critère d'arrêt.

Le coefficient θ_2 négatif s'explique par la forte corrélation entre surface et nombre de pièces : à surface égale, un logement découpé en plus de pièces ne vaut pas plus cher.

Prédiction. Le nouveau logement doit subir **la même normalisation** que les données d'apprentissage : on utilise `scaler.transform` (et non `fit_transform`, qui recalculerait moyenne et écart-type), puis on ajoute le 1 correspondant à θ_0 .

```
In [19]: nouveau = np.array([[1650, 3]]) # surface 1650, 3
pièces
nouveau_norm = scaler.transform(nouveau) # même normalisation
que l'apprentissage
x_nouveau = np.hstack([np.ones((1, 1)), nouveau_norm])
prix = h(theta_opt, x_nouveau).item()
print('logement normalisé :', nouveau_norm.round(4))
print(f'prix prédit : {prix:,.0f}'.replace(',', ' '))
print(f'(avec la solution exacte : {h(theta_exact,
x_nouveau).item():,.0f}'.replace(',', ' '))
```

```
logement normalisé : [[-0.446 -0.2261]]
prix prédit : 293 539
(avec la solution exacte : 293 081)
```

Réponse : le prix prédit pour un logement de 1650 m² et 3 pièces est d'environ **293 000** (293 539 avec le θ obtenu par descente de gradient, 293 081 avec la solution exacte).

Remarque : dans ce jeu de données (Portland), les surfaces sont en réalité exprimées en pieds carrés (ft²) ; on garde l'unité de l'énoncé, le calcul est identique.